

MODULE 5

Chapter 1

Next-generation databases are a class of databases designed to address the limitations and requirements of modern applications, such as scalability, flexibility, and real-time data processing.

They often employ non-relational (NoSQL) or distributed architectures to provide better performance and scalability compared to traditional relational databases.

Next-generation databases prioritize characteristics like horizontal scalability, schema flexibility, and support for unstructured or semi-structured data, which may not be well-suited for traditional relational databases.

1. **Performance:** Next-generation databases often provide superior performance compared to traditional databases, thanks to optimized data storage, indexing, and query execution techniques. They are optimized for high throughput and low-latency data access, enabling faster response times for applications.
2. **Flexibility:** Next-generation databases support flexible data models and schema designs, allowing organizations to store and manage diverse types of data, including structured, semi-structured, and unstructured data. They can adapt to changing data requirements and evolving business needs more effectively than traditional databases.
3. **High Availability:** Next-generation databases prioritize data availability and reliability, employing built-in replication, failover mechanisms, and distributed storage architectures. They ensure that data remains accessible

even in the event of hardware failures, network issues, or other disruptions, minimizing downtime and data loss.

4. **Real-time Analytics:** Next-generation databases support real-time analytics and streaming data processing, enabling organizations to analyze and derive insights from rapidly changing data streams. They offer features for continuous data ingestion, processing, and analysis, allowing for timely decision-making and actionable insights.
5. **Security:** Next-generation databases incorporate advanced security features to protect sensitive data and ensure regulatory compliance. They offer encryption, access control, audit logging, and other security mechanisms to safeguard data against unauthorized access, data breaches, and other security threats.
6. **Cost-effectiveness:** Many next-generation databases offer cloud-native deployment options and pay-as-you-go pricing models, reducing infrastructure costs and operational overhead for organizations. They allow organizations to scale resources dynamically based on usage, minimizing upfront investments and maximizing cost-efficiency.

Examples of next-generation databases include:

1. **MongoDB:** A document-oriented database that stores data in JSON-like documents, MongoDB is commonly used for applications requiring flexible schema design and scalability.
2. **Apache Cassandra:** A distributed, wide-column store database designed for high availability and linear scalability, Apache Cassandra is used for real-time data processing, IoT applications, and time-series data storage.

3. **Amazon DynamoDB:** *A fully managed NoSQL database service provided by AWS, DynamoDB offers seamless scalability, low-latency data access, and built-in support for multi-region replication, making it suitable for web-scale applications and real-time analytics.*
4. **Elasticsearch:** A distributed, RESTful search and analytics engine, Elasticsearch is commonly used for full-text search, log analytics, and real-time data visualization in applications requiring fast and scalable search capabilities.

DISTRIBUTED DATABASE

A distributed database is a collection of multiple interconnected databases spread across different geographical locations and managed by a central control module.

Unlike a centralized database, where all data is stored in a single location, a distributed database distributes data across multiple sites or nodes, allowing users and applications to access and manipulate data as if it were stored in a single logical database.

Distributed database management systems (DDBMS) are used in a wide range of applications across various industries due to their ability to provide scalability, high availability, and fault tolerance. Here are some applications that leverage DDBMS:

E-commerce Platforms: E-commerce websites and applications often use distributed databases to handle large volumes of transactions, user data, and inventory management across multiple regions. For example, Amazon's e-commerce platform relies on distributed databases to ensure that product information, customer data,

and order processing are available and consistent across different geographic locations.

Social Media Platforms: Social media platforms like Facebook, Twitter, and Instagram employ distributed databases to manage vast amounts of user-generated content, interactions, and relationships. These systems need to support millions or even billions of users while providing real-time updates and high availability.

Financial Services: Banks, stock exchanges, and financial institutions use distributed databases to manage transactions, customer accounts, and financial data across branches or data centers. Distributed systems ensure that financial transactions are processed quickly, reliably, and securely, even during peak times or in the event of failures.

Advantages:

1. **Scalability:** Distributed databases can scale horizontally by adding new nodes, allowing them to accommodate increasing data volumes and user demands more effectively than centralized databases.
2. **Fault Tolerance:** Distributed databases offer improved fault tolerance and availability by replicating data across multiple nodes. In the event of node failures or network partitions, data remains accessible from other nodes, ensuring uninterrupted service.
3. **Improved Performance:** Distributed databases can enhance performance by distributing query processing and data access across multiple nodes, reducing response times and network latency, especially for geographically distributed users.
4. **Data Localization:** Distributing data closer to users or applications can improve data access speeds and reduce network bandwidth consumption, particularly in scenarios with distributed user bases.

Challenges:

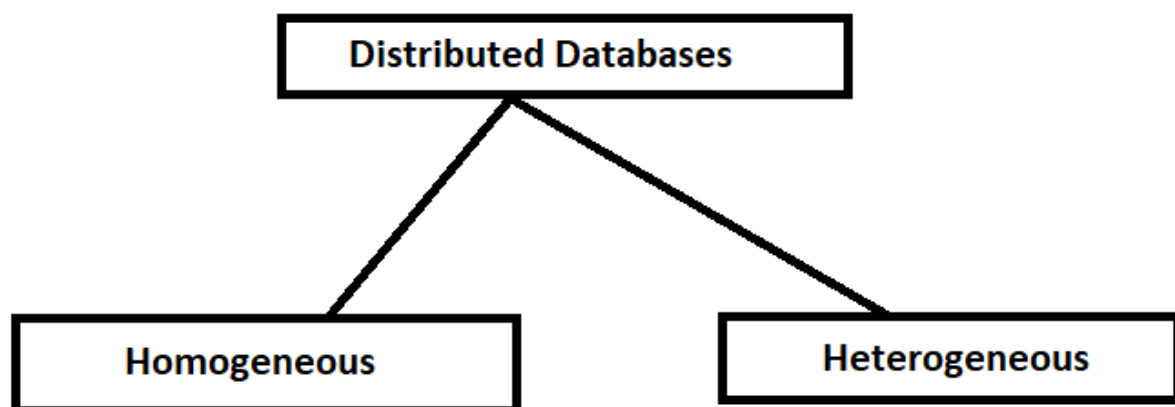
1. **Data Consistency:** Ensuring data consistency across distributed nodes can be challenging due to network latency, node failures, and concurrent updates. Maintaining data consistency while allowing for scalability and fault tolerance requires sophisticated replication and consistency management mechanisms.
2. **Transaction Management:** Coordinating distributed transactions across multiple nodes while ensuring ACID properties (Atomicity, Consistency, Isolation, Durability) presents challenges such as distributed deadlock detection, distributed concurrency control, and two-phase commit protocols.
3. **Network Complexity:** Distributed databases introduce additional network complexity and communication overhead for data transfer, transaction coordination, and synchronization, requiring robust network infrastructure and protocols to manage distributed operations efficiently.

Examples:

- **Google Spanner:** Google Spanner is a globally distributed database service that provides strong consistency, horizontal scalability, and high availability across multiple data centers worldwide.
- **Amazon DynamoDB:** Amazon DynamoDB is a fully managed NoSQL database service that offers seamless scalability and high availability by distributing data across multiple Availability Zones (AZs) within AWS regions. It provides automatic replication, consistent performance, and low-latency access to data for applications with global user bases.

Homogeneous and Heterogeneous Databases

Distributed databases can be classified into homogeneous and heterogeneous databases. Here we will discuss Homogeneous and Heterogeneous databases.



Types of Distributed Database

Homogeneous Databases

- ✓ Homogeneous databases are characterized by all participating nodes sharing the same database management system (DBMS) and schema structure.
- ✓ These databases are designed to offer a unified and consistent view of data across all nodes.

Let's go to the characteristics and steps involved in establishing a homogeneous database.

Characteristics of Homogeneous Databases

1. **Uniformity:** All nodes utilize the same DBMS software and possess identical database schemas.
2. **Data Consistency:** Changes made to the database on one node are automatically propagated to other nodes, ensuring data consistency.
3. **Simplicity:** Homogeneous databases are relatively easier to manage as the same DBMS software is employed throughout the system.

Steps to Set up a Homogeneous Database:

Select a DBMS: Choose a DBMS that aligns with the distributed system's requirements, such as MySQL, PostgreSQL, or Oracle.

Install the DBMS: Install the chosen DBMS on each node within the distributed system.

Design the Schema: Create a unified database schema to be shared across all nodes.

1. **Establish Communication:** Configure network connectivity between the nodes to facilitate data replication and synchronization.
2. **Implement Replication:** Set up replication mechanisms provided by the DBMS to ensure changes made on one node are propagated to others.

Use Case Example

Consider a distributed e-commerce system with multiple nodes handling customer orders, inventory management, and shipping. Employing a homogeneous database approach, all nodes share the same DBMS (e.g., MySQL) and adhere to a consistent schema.

When an order is placed on one node, the system automatically synchronizes the order details and inventory updates across all nodes, enabling real-time visibility and consistency.

Heterogeneous Databases

Contrary to homogeneous databases, heterogeneous databases allow nodes in a distributed system to use different DBMS software or possess varying database schemas. This approach caters to diverse requirements and facilitates seamless integration between nodes employing different technologies. Let's explore the characteristics and steps involved in setting up a heterogeneous database.

Characteristics of Heterogeneous Databases

- **Flexibility:** Nodes can employ different DBMS software, such as MySQL, MongoDB, or Cassandra, based on their specific needs.

- **Schema Mapping:** Heterogeneous databases necessitate mapping between different schemas to ensure interoperability between nodes.
- **Data Transformation:** Data might need to be transformed or translated between different formats or encodings to maintain consistency.

Steps to Set up a Heterogeneous Database

- **Identify Diverse Requirements:** Understand the specific needs of each node in the distributed system and select the appropriate DBMS software accordingly.
- **Define Schema Mapping:** Analyze the differences in database schemas between nodes and establish mapping rules to convert data between schemas.
- **Implement Data Transformation:** Develop mechanisms or scripts to transform data from one format to another, ensuring seamless integration.
- **Establish Communication:** Configure network connectivity and establish communication channels between heterogeneous nodes.

Use Case Example

Imagine a distributed system where one node utilizes a traditional relational database (e.g., MySQL) for order management, while another node relies on a NoSQL database (e.g., MongoDB) for user analytics. Employing a heterogeneous database approach enables the two nodes to leverage their preferred DBMS technologies while facilitating data exchange through schema mapping and data transformation.

Examples of homogeneous and heterogeneous databases:

Examples:

Homogeneous Distributed Database: Example: Oracle Real Application Clusters (RAC)

Oracle RAC is a feature of Oracle Database that allows multiple instances to access a single database simultaneously. It provides a scalable and highly available solution by distributing the workload across multiple nodes in a cluster. Each node in the cluster runs the same Oracle Database software and stores the same data, ensuring consistency and uniformity across the distributed environment.

Heterogeneous Distributed Database: Example: Data Warehousing Environment

In a data warehousing environment, data may be distributed across heterogeneous databases that use different DBMS software and data models. For instance, an organization may use a combination of relational databases, NoSQL databases, and legacy systems to store and manage different types of data. To integrate and analyze data from these disparate sources, data warehousing solutions employ techniques such as data extraction, transformation, and loading (ETL) processes, as well as federated query mechanisms to access data across heterogeneous databases.

These examples illustrate how distributed databases can be either homogeneous or heterogeneous, depending on the uniformity or diversity of the underlying

DBMS software and data models used across different sites in the distributed environment.

Data fragmentation is a concept in database management systems (DBMS) where a large dataset is divided into smaller, more manageable pieces or fragments. These fragments can be distributed across different locations or nodes in a distributed environment, contributing to improved data storage, access, and management. Here's an explanation of the concept of data fragmentation along with its different types:

Types of Data Fragmentation:

a. **Horizontal Fragmentation:**
Horizontal fragmentation involves dividing a table's rows into subsets or fragments based on a condition or attribute value. Each fragment contains a subset of rows that satisfy the fragmentation condition.
Example: In a customer database, customer records may be horizontally fragmented based on geographic regions, where each fragment contains customers from a specific region.

Example: Horizontal Data Fragmentation

Consider a company's customer database that stores information about customers' orders, such as customer name, order ID, product ID, and quantity. Due to business requirements, the company decides to horizontally fragment the database based on geographical regions to improve data access and performance for regional sales teams.

Original Table:

Customer Name	Order ID	Product ID	Quantity
John Doe	1001	101	2
Jane Smith	1002	102	1
Bob Johnson	1003	103	3
Alice Brown	1004	101	1
...	...	...	...

Fragmented Tables:

1. North America Fragment:			
• Stores data for customers located in North America.			
Customer Name	Order ID	Product ID	Quantity
John Doe	1001	101	2

Customer Name	Order ID	Product ID	Quantity
Jane Smith	1002	102	1
...	...	...	...

2. Europe Fragment:

- Stores data for customers located in Europe.

Customer Name	Order ID	Product ID	Quantity
Bob Johnson	1003	103	3
Alice Brown	1004	101	1
...	...	...	...

By horizontally fragmenting the database based on geographical regions, the company can optimize data access and retrieval for sales teams operating in different regions. Each fragment contains a subset of the original data that is relevant to a specific region, reducing data transfer overhead and improving query performance for localized operations.

Data fragmentation allows for better scalability, improved performance, and enhanced data management in distributed or partitioned database environments.

b. Vertical Fragmentation:

Vertical fragmentation involves dividing a table's columns into subsets or fragments. Each fragment contains a subset of columns for all rows in the table. Example: In a product catalog database, product information may be vertically fragmented based on product categories, where each fragment contains columns specific to a particular category.

Example: Vertical Data Fragmentation

Consider a company's employee database that stores information about employees, including their personal details, salary information, and department. Due to security requirements, the company decides to vertically fragment the employee table into two fragments: one containing personal details and another containing salary information.

Original Table:

Employee ID	First Name	Last Name	Date of Birth	Salary	Department
1001	John	Doe	1985-05-15	50000	Sales
1002	Jane	Smith	1990-08-20	60000	Marketing
1003	Bob	Johnson	1978-12-10	70000	Finance
1004	Alice	Brown	1982-03-25	55000	HR
...	...	...	...	...	...

Fragmented Tables:

1. Personal Details Fragment:

- Contains columns related to personal details such as employee ID, first name, last name, and date of birth.

Employee ID	First Name	Last Name	Date of Birth
1001	John	Doe	1985-05-15
1002	Jane	Smith	1990-08-20
...	...	...	...

2. Salary Information Fragment:

- Contains columns related to salary information such as employee ID and salary.

Employee ID	Salary
1001	50000
1002	60000
...	...

By vertically fragmenting the employee table, the company can enforce stricter access controls by storing sensitive salary information separately from personal details. This separation can help improve data security and privacy compliance. Additionally, it can optimize query performance by reducing the size of each fragment and minimizing data transfer overhead.

Vertical fragmentation allows for better data management, improved security, and enhanced performance in database systems, especially in scenarios where specific subsets of columns need to be accessed or protected separately.

c. Hybrid Fragmentation:

Hybrid fragmentation combines elements of both horizontal and vertical fragmentation.

It involves dividing a table into subsets of rows and columns based on specific criteria.

Example: In an inventory management system, inventory data may be fragmented horizontally based on warehouse locations and vertically based on product categories, creating hybrid fragments.

Advantages and challenges of using horizontal and vertical data fragmentation in database management systems

Advantages:

Improved Performance: Horizontal fragmentation distributes data across multiple nodes, enabling parallel processing and improving query performance, especially for distributed queries that access specific subsets of data.

Scalability: It facilitates horizontal scaling by allowing new nodes to be added to accommodate increasing data volumes, providing scalability without affecting existing data distribution.

Data Localization: Fragmenting data based on criteria such as geographic regions or customer segments enables data to be localized closer to the users or applications that frequently access it, reducing network latency and improving data access speeds.

Challenges:

Data Distribution Overhead: Managing and distributing data fragments across multiple nodes can introduce overhead in terms of data distribution, synchronization, and coordination, especially as the dataset grows.

Data Consistency: Ensuring data consistency across distributed fragments can be challenging, as updates or modifications to data may need to be propagated and synchronized across multiple nodes to maintain consistency.

Example: Horizontal fragmentation of a customer database based on geographic regions can improve query performance and data access speeds for users located in different regions. However, ensuring consistency of customer data across distributed fragments may require synchronization mechanisms and careful management of updates.

Vertical Data Fragmentation:

Advantages

Reduced Data Redundancy: Vertical fragmentation eliminates redundant data by storing only relevant columns in each fragment, reducing storage requirements and improving resource utilization.

Customized Access Control: It allows for fine-grained access control by partitioning sensitive or confidential data into separate fragments, enabling stricter access controls and data security.

Optimized Data Access: Fragmenting data based on usage patterns or data categories can optimize data access and retrieval by storing related columns together, reducing the amount of data accessed for each query.

Challenges:

Increased Joins: Vertical fragmentation may result in increased join operations for queries that require data from multiple fragments, leading to higher query processing times and potentially complex query optimization challenges.

Dependency Management: Managing dependencies between fragmented data, especially when data from different fragments is required for transactional consistency or referential integrity, can be complex and require careful coordination.

Example: Vertical fragmentation of a product catalog database based on product categories allows for efficient storage and retrieval of product

information. However, queries that involve retrieving data from multiple product categories may require join operations, impacting query performance.

Two different ways to store data using distributed data storage technique.

1. **Replication** - With this strategy, every aspect of the connection is redundantly kept at two or more locations. It is a completely redundant database if the entire database is accessible from every location. Systems preserve copies of the data as a result of replication. This has advantages since it makes more data accessible at many locations. Moreover, query requests can now be handled in parallel. But, there are some drawbacks as well. Data must be updated often. All changes performed at one site must be documented at every site where that relation is stored in order to avoid inconsistent results. There is a tone of overhead here. Moreover, since concurrent access must now be monitored across several sites, concurrency management becomes far more complicated.
2. **Fragmentation** - In this method, the relationships are broken up into smaller pieces and each fragment is kept in the many locations where it is needed. To ensure there is no data loss, the pieces must be created in a way that allows for the reconstruction of the original relation. As fragmentation doesn't result in duplicate data, consistency is not a concern.

Chapter 2

Analyse the concept of an object-based database (OODB) and how it differs from traditional relational databases.

- An object-based database (OODB) is a type of database management system that extends the relational model to include support for complex data types and object-oriented programming concepts. Unlike traditional relational databases that store data in tables with rows and columns,
- OODBs allow for the storage and manipulation of more complex data structures, such as objects, arrays, and nested structures.
- OODBs support encapsulation, inheritance, and polymorphism, which are fundamental principles of object-oriented programming, allowing for more expressive and flexible data modeling compared to relational databases.

Challenges and considerations in migrating from a relational database to an object-based database include:

1. **Data Model Mapping:** Mapping relational data models to object-based data models may require significant schema redesign and data migration efforts to accommodate differences in data modeling paradigms.
2. **Query Language and APIs:** Applications built on relational database technologies may require modifications to work with object-based databases, including changes to data access logic, query patterns, and transaction management.
3. **Performance and Scalability:** Object-based databases may offer improved performance and scalability for certain types of applications but may require performance tuning and optimization efforts to achieve optimal performance in specific use cases.
4. **Tooling and Ecosystem:** Object-based databases may have different tooling and ecosystem compared to relational databases, requiring adjustments to development, deployment, and monitoring practices.
5. **Training and Skills:** Migrating to an object-based database may require training and upskilling of database administrators, developers, and other stakeholders to familiarize them with object-oriented data modeling concepts and best practices.

So we can conclude that migrating from a relational database to an object-based database involves various technical, operational, and organizational challenges that require careful planning, assessment of requirements, and consideration of the potential benefits and trade-offs of databases adopting object-based database technologies.

Structured types in SQL allow users to define custom data types composed of multiple attributes or fields. These attributes can be of primitive data types (such as integers, strings, or dates) or other structured types, enabling the creation of complex data structures.

Structured types are typically used to model entities or objects in the real world with multiple properties and relationships. They provide a way to encapsulate related data and behaviors into a single unit, promoting data integrity and abstraction.

For example, a structured type **Employee** may have attributes such as **name**, **age**, and **department**, representing properties of an employee entity.

- Inheritance in SQL allows structured types to inherit attributes and behaviors from other structured types, enabling the reuse and extension of existing data models.
- Inheritance relationship is similar to object-oriented programming concepts, where subclasses inherit properties and methods from their parent classes. In SQL, inheritance is typically achieved using subtype relationships between structured types.
- Subtypes inherit attributes and methods from their supertypes and may add additional attributes or override existing ones. This allows for a hierarchical organization of data models, where more specific types can be defined as extensions of more general types.

For example, a structured type **Manager** may inherit attributes from the **Employee** type and add additional attributes such as **manager_level** and responsibilities

For example illustrating the use of structured types and inheritance in SQL, including the creation of structured types, inheritance relationships, and querying of subtype-specific attributes.

In this example, we define a structured type **Employee_Type** with attributes **employee_id**, **name**, and **hire_date**. We then define a subtype **Manager_Type** inheriting from **Employee_Type** and adding additional attributes **manager_level** and **responsibilities**. Tables are created for storing employees and managers, and data is inserted into these tables. Finally, we query the **Managers** table to retrieve subtype-specific attributes.

Example structured type for Employee

```
CREATE TYPE Employee_Type AS OBJECT (  
    employee_id INT,  
    name VARCHAR(50),  
    hire_date DATE );  
/-- Define a subtype Manager inheriting from Employee  
  
CREATE TYPE Manager_Type UNDER Employee_Type (  
    manager_level VARCHAR(20),  
    responsibilities VARCHAR(100));  
/  
-- Create tables for storing employees and managers  
  
CREATE TABLE Employees OF Employee_Type;  
  
CREATE TABLE Managers OF Manager_Type;  
-- Insert data into the tables
```

```
INSERT INTO Employees VALUES (101, 'John Doe', '2022-01-01');
```

```
INSERT INTO Managers VALUES (201, 'Jane Smith', '2022-02-01', 'Senior', 'Team Management');
```

--

Querying subtype-specific attributes

```
SELECT * FROM Managers;
```

Inheritance in SQL databases refers to a mechanism that allows one database object (such as a table or a view) to inherit properties and attributes from another database object. This concept is similar to inheritance in object-oriented programming, where subclasses inherit characteristics from superclasses. In SQL, inheritance is typically implemented using features like table inheritance or views.

Table Inheritance:

In table inheritance, a child table inherits attributes and constraints from a parent table, allowing it to reuse the structure and properties of the parent table. Child tables inherit columns, data types, constraints (such as primary keys and foreign keys), and other properties from their parent table.

Any modifications made to the parent table schema are automatically reflected in the child tables, ensuring consistency and synchronization.

```
CREATE TABLE person (  
    person_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50)  
);
```

```
CREATE TABLE employee (  
    employee_id INT PRIMARY KEY,  
    position VARCHAR(50),  
    -- Other employee-specific columns  
) INHERITS (person);
```

```
CREATE TABLE customer (  
    customer_id INT PRIMARY KEY,  
    membership_level VARCHAR(50),  
    -- Other customer-specific columns ) INHERITS (person);
```

In this example, the **employee** and **customer** tables inherit the **person_id**, **first_name**, and **last_name** attributes from the **person** table.

ast_name columns from the **person** table, along with any constraints defined on those columns.

Example:

Consider a scenario where we have a parent table called **person** and two child tables called **employee** and **customer**. The **employee** table inherits from the **person** table, as employees are a subset of persons. Similarly, the **customer** table also inherits from the **person** table.

Module 5

MongoDB is a popular document-oriented NoSQL database that provides a flexible and scalable solution for storing and managing unstructured or semi-structured data. Unlike traditional relational databases, which organize data into tables with fixed schemas, MongoDB stores data in flexible JSON-like documents within collections.

Here are some **key features** of MongoDB compared to traditional relational databases:

1. **Flexible Schema:** MongoDB uses a flexible schema model, allowing documents within the same collection to have different structures and fields. This flexibility enables easier adaptation to changing data requirements and eliminates the need for costly schema migrations.
2. **Document-Oriented Storage:** MongoDB stores data in BSON (Binary JSON) documents, which support nested structures, arrays, and other complex data types. This document-oriented storage model is well-suited for representing hierarchical and deeply nested data structures.
3. **Scalability:** MongoDB is designed for horizontal scalability, allowing for distributed deployment across multiple nodes and clusters. It supports sharding, replication, and auto-scaling features, enabling seamless scalability as data volumes and workload demands increase.
4. **High Performance:** MongoDB offers high-performance data access and query processing capabilities, with support for indexing, aggregation pipelines, and parallel query execution. It utilizes in-memory caching and efficient storage mechanisms to optimize data retrieval speeds and query performance.
5. **Rich Query Language:** MongoDB provides a powerful query language and aggregation framework that supports a wide range of query operations, including CRUD operations (Create, Read, Update, Delete), filtering, sorting, grouping, and aggregation.
6. **Geospatial Indexing:** MongoDB includes support for geospatial indexing and queries, allowing for efficient storage and retrieval of location-based data, such as GPS coordinates and spatial geometries.

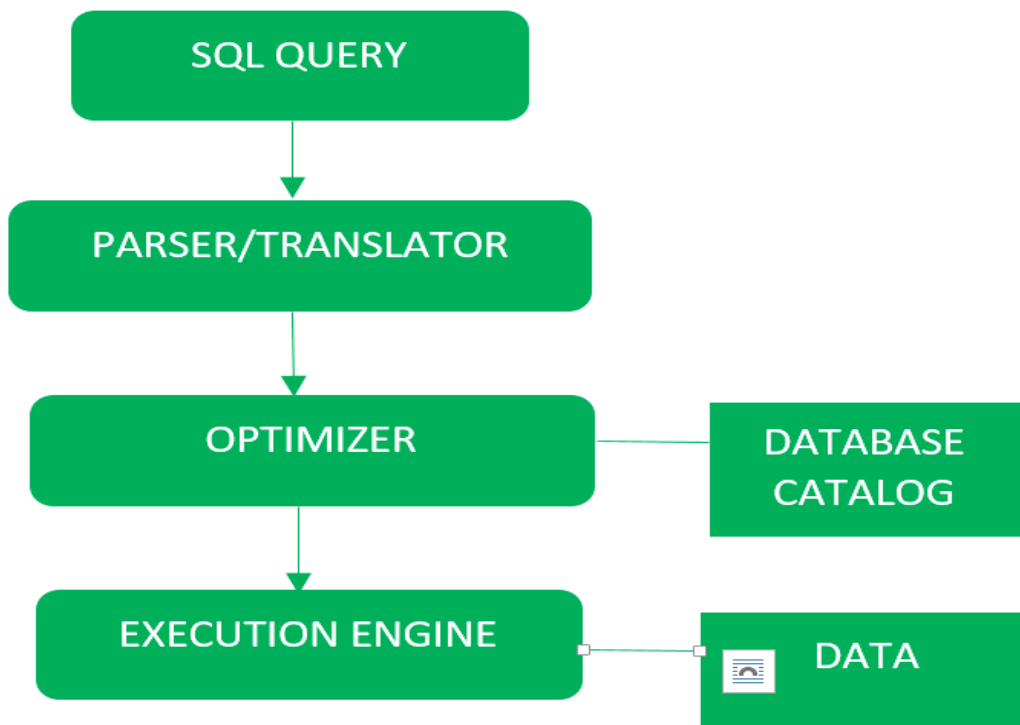
7. **Fault Tolerance and High Availability:** MongoDB ensures data durability, fault tolerance, and high availability through features such as automatic failover, data replication, and distributed data storage. It supports multi-region replication and data center awareness for disaster recovery and fault tolerance.

In summary, MongoDB offers a flexible, scalable, and high-performance solution for managing diverse and evolving data requirements, making it well-suited for modern web applications, real-time analytics, content management systems, and other data-intensive applications.

SQL | Query Processing

Query Processing includes translations on high level Queries into low level expressions that can be used at physical level of file system, query optimization and actual execution of query to get the actual result.

Block Diagram of Query Processing is as:



Query processing in SQL involves several steps that the database management system (DBMS) follows to execute a SQL query efficiently

and return the desired results. Here's an overview of the query processing steps in SQL:

Query Parsing:

1. The first step in query processing is parsing, where the DBMS parses the SQL query to check its syntax and structure. If the query is syntactically correct, it proceeds to the next steps. If there are any syntax errors, an error message is generated.

2. **Query Optimization:**

After parsing, the DBMS performs query optimization to generate an efficient query execution plan. The optimizer analyzes various execution plans and selects the one that minimizes the query's cost based on factors such as index usage, join order, and access methods. This step aims to reduce query processing time and resource utilization.

3. **Query Execution:**

Once the query optimizer selects the execution plan, the DBMS executes the query plan to retrieve or manipulate data from the database tables. This involves accessing data stored on disk, applying filters and conditions specified in the query, performing joins, aggregations, and sorting as necessary.

4. **Data Retrieval:**

During query execution, the DBMS retrieves data from the database tables based on the specified query criteria. This may involve scanning entire tables, using indexes to locate specific rows efficiently, or performing other access methods such as table joins and subqueries.

5. **Result Processing:**

As the data is retrieved, the DBMS processes it according to the query requirements. This may involve filtering rows based on WHERE clause conditions, performing aggregate functions (e.g., SUM, AVG, COUNT), grouping data, and sorting the result set.

6. **Result Presentation:**

Once the query execution is complete and the result set is processed, the DBMS presents the final result to the user or application. This may involve formatting the result set in a tabular format, applying any specified formatting or ordering, and returning it to the client for further processing or display.

7. **Transaction Management:**

In parallel with query processing, the DBMS manages transactions to ensure data consistency and integrity. This involves maintaining transaction logs, enforcing ACID properties (Atomicity, Consistency, Isolation, Durability), and handling transaction commit or rollback operations.

Overall, query processing in SQL involves a series of steps, including parsing, optimization, execution, data retrieval, result processing, presentation, and

transaction management. Each step is crucial for efficiently processing SQL queries and returning accurate results from the database.

Query Processing Example

Let's consider an example to show you the query processing steps. Suppose we have a database with a table "student". It contains the rollno, first_name, last_name, and CGPA. The following SQL query is used to retrieve the names of all students whose mark is greater than 50:

```
SELECT first_name, last_name FROM student WHERE mark > 50;
```

Now let us understand how this query will give us results by following the query processing steps.

Parsing: Firstly, the query will be parsed. This will also check whether the syntax is correct or not. Then this query will be converted into a parse tree. This tree will look like this

```
SELECT
|
+-- first_name
|
+-- last_name
FROM
|
+-- student
WHERE
|
+-- mark
    |
    +-- >
        |
        +-- 50
```

Optimization: The DBMS determines the most efficient way to execute the query by considering factors such as whether an index exists on the mark field. In this case, the DBMS might use an index on the mark field to efficiently retrieve the matching rows.

Evaluation: The DBMS executes the optimized query. It retrieves the results from the database. Then it returns the first_name and last_name of all students whose evaluation is greater than 50.